

Appendix: Examples of Bug Triggering Code Uncovered by Mutators of TyMut

ANONYMOUS AUTHOR(S)

This document is a supplementary file of our paper submitted to OOPSLA-2025, i.e., *Fuzzing C++ Compilers via Type-Driven Mutation*. This document presents detailed confirmed compiler bugs identified by mutation operators of TyMut on GCC and LLVM, with 1 example code for each mutation operator. Please note that mutation operators that have not currently triggered any bug(e.g. **C3[✓]. Integer Arrayization**), or the uncovered bugs have not been confirmed(e.g. **I8[✗]. Multi-Inheritance Construction**), are not included.

ACM Reference Format:

Anonymous Author(s). 2025. Appendix: Examples of Bug Triggering Code Uncovered by Mutators of TyMut. *Proc. ACM Program. Lang.* 1, OOPSLA, Article XX (October 2025), 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Inheritance Mutation Operators

The mutation operators manipulate inheritance structures or functions along the inheritance chain.

I1[✓]. Top Empty Base Insertion. Given a program p with its inheritance graph $\mathcal{G}$, this mutation operator modifies the inheritance structure by inserting an empty class to the top-most class of a *complete path* in $\mathcal{G}$. A *complete path* is a sequence of classes where no other class in $\mathcal{G}$ can serve as a base or a derived class for any class within the path. This operator is well-formed.

Listing(1) shows a bug uncovered by this mutator. In this example code, the mutated inheritance graph $\mathcal{G}'$ is NoMut1 $\rightarrow$ NoMut3. When empty is inserted to the top of $\mathcal{G}'$, the constexpr variable handling module of GCC starts to crash in line 7 when parsing NoMut3.

I2[✗]. Middle Empty Base Insertion. Given a program p with its inheritance graph $\mathcal{G}$ and environment mapping Γ , this mutation operator modifies the inheritance structure by inserting an empty class into the middle of a *complete path* in $\mathcal{G}$. The inheritance accessible modifier of the inserted class is randomly marked as public, private, or protected. This operator may change the accessibility set of its derived classes, and thus is not-well-formed.

Listing (2) shows a bug uncovered by this mutator. In this example code, the mutator inserts class mut into the middle of class F and class E, making class E have both an empty direct base class and a non-empty indirect base class. The __built_bit_cast function in line 11 does not expect classes like this to be passed as an argument, which causes LLVM to crash.

I3[✗]. Inheritance Modifier Replacement.

This mutation operator changes the inheritance modifier to another (e.g., changing public inheritance to private inheritance). This operator may affect the accessibility set of classes and thus is not-well-formed.

Listing(3) shows a bug uncovered by this mutator. In this example code, the mutator changes the inheritance modifier of struct X with struct A from public to private, which makes the static_cast expression in line 6 invalid. LLVM does not correctly notice that by accepting the invalid code.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2475-1421/2025/10-ARTXX

<https://doi.org/XXXXXXX.XXXXXXX>

```

1  struct empty{}; //inserted
2  struct NoMut1 : empty{ int a, b; }; //replaced
3  struct NoMut3 : NoMut1 {
4      constexpr NoMut3(int a, int b){}
5  };
6  void mutable_subobjects() {
7      constexpr NoMut3 nm3 = {1, 2};
8      struct A{
9          void f() {
10             static_assert(nm3.a == 1, "");
11         }
12     };
13 }

```

Listing 1. GCC-11XX93, crash.

```

1  class F {
2  public:
3      char f[2];
4  };
5  class mut : public F{}; //inserted
6  class E : public mut //replaced
7  {
8  public:
9      constexpr E() : F{3,4} {}
10 };
11 static_assert(__builtin_bit_cast(short, E()) == (LITTLE_END ?
12             578437695752307201 : 72623859790382856));

```

Listing 2. LLVM-13XX57, crash.

```

1  struct A { int a;};
2  struct X : private A{}; //replaced
3
4  int main() {
5      X x;
6      int a=static_cast<A&&>(x).a;
7  }

```

Listing 3. LLVM-12XX29, accepts-invalid.

I4^X. Normal Inheritance Virtualization. Given a complete path in $\mathcal{G}$, if all classes in the path do not participate in multiple inheritance (i.e., each node has at most one incoming and one outgoing edge), this mutation operator randomly converts a normal inheritance relationship to a virtual inheritance (e.g., the change edge label from $\sqsubseteq^+$ to $\sqsubseteq_v^+$). This operator is not-well-formed.

Listing(4) shows a bug uncovered by this mutator. In this example code, the mutator chooses to convert the inheritance relationship between derived and base to a virtual inheritance. As specified in C++ standard, a const explicit copy-assign operator is not allowed to be used in a virtual derived class, as well as copy constructors and move constructors. GCC correctly rejects the latter, but forgets to reject copy-assign operators. Note that this code can be rejected by our well-formedness rule: **VIRTUAL-INHERITANCE-CONST**.

I7^X. Function Finalization. Given a class with a virtual function (i.e., marked with the `virtual` modifier), this mutation operator changes it to a final function. In C++, final functions are closed to

```

1  struct base { };
2  struct derived : virtual base //replaced
3  {
4      constexpr derived& operator=(derived const&) = default;
5  };

```

Listing 4. GCC-11XX83, accepts-invalid.

```

1  template<class T, class = decltype(T())>
2  char f(int);
3
4  struct A {
5      virtual ~A() = 0 final; //inserted
6  };
7
8  auto a = f<A>(&);

```

Listing 5. LLVM-12XX39, crash.

```

1  template <typename=int> //replaced
2  struct D{};
3
4  extern D d;

```

Listing 6. GCC-11XX82, accepts-invalid.

overriding, while virtual functions are designed to be overridden. Thus, this operator often leads to invalid programs, and it is not-well-formed.

Listing(5) shows a bug uncovered by this mutator. In this example code, the mutator constructs an invalid mutated program by incorrect inserts a `final` specifier behind a pure virtual function , which causes the syntax parses of LLVM to crash.

2 Template Generic Mutation Operators

Template mutation operators manipulate the structure of the template declaration.

G1^{*}. Generic Narrowing. Given a generic class $cname<\tau>$, this mutation operator replaces τ to τ' , where τ' is a subtype according to $\mathcal{L}$. This operator may introduce inconsistency of the invocation of its ctors and thus is not-well-formed.

Listing(6) shows a bug uncovered by this mutator. In this example code, the mutator replaces the generic type of struct `D` from `T` to `(K : int)`. The instantiation statement in line 4 will then be invalid because the template argument specifier must not be eliminated when the instantiated object is an extern class. GCC does not correctly notice that.

G3[✓]. Function Templatization. Given a member function of a class, this operator inserts a declaration of the function by adding a template generic. This operator is well-formed.

Listing(7) shows a rejects-valid bug uncovered by this mutator. In this example code, the mutator inserts a declaration of template member function `A::f` in line 6, which provides an overloaded function of void `f(T)` in line 4. The compiler should choose the original void `f(T)` when parsing the template instantiation statement in line 9, while GCC ends up selecting the newly-inserted function and incorrectly rejecting the code.

```

1  template <class T>
2  struct A
3  {
4      void f(T) {}
5      template <typename MUT> void f(MUT t){} //inserted
6  };
7
8  template
9  void A<int>::f(int);

```

Listing 7. GCC-11XX51, rejects-valid.

```

1  constexpr void foo (int&& i) { i = 0; }
2
3  void bar(int&& i)
4  {
5      foo(reinterpret_cast<int&&>(i)); //replaced
6  }

```

Listing 8. GCC-11XX56, rejects-valid.

3 Type Convention Mutation Operators

Type Convention plays an important role in the type systems of programming languages. We use the following mutation operators to modify the types.

C1[✓]. Casting Form Converting. Given a dynamic or static casting, this mutation operator replaces it with a reinterpret casting, which is equivalent to the C-style casting without any type safety checks. This operator is well-formed since the seed programs are guaranteed to be valid.

Listing(8) shows a bug uncovered by this mutator. In this example code, the mutator replaces the original `static_cast` operator in line 6 to `reinterpret_cast`, which unexpected makes the code rejected by GCC. It ends up that GCC would incorrectly reject `reinterpret_cast` expressions when the source type and the target type are both rvalue reference.

C2[✓]. Casting Overloading. Given a class, this mutation operator inserts useless overloading conversion operators for primary types. In a class declaration, its overloaded conversion operators are special functions that enable converting the class to the target type. This operator is well-formed.

Listing(9) shows a bug uncovered by this mutator. In this example code, the mutator inserts an overloading conversion operator `operator int` inside class `X`, and tries to use it as a default argument value in line 6. GCC incorrectly rejects that by erroneously disallowing the friend function `friend_set` of class `X` to call its private conversion operator.

C4[✓]. Order Lifting. Given an expression of a primary type, this mutation operator lifts the expression to its corresponding function type. Formally, given an expression e , it produces its lambda abstraction with no parameters: $\lambda().e$. This operator lifts the type from the primary type $\llbracket e \rrbracket$ to the function type $\top \rightarrow \llbracket e \rrbracket$. This operator is well-formed.

Listing(10) shows a bug uncovered by this mutator. In this example code, the mutator lifts the type of the value of the function parameter `a` in line 3 from integer type to a lambda expression. GCC does not expect the default argument value of a friend function to be a lambda expression, which results in a crash.

4 Accessibility Mutation Operators

The accessibility of class members in C++ is an integral part of its type system, since it restricts which members can be accessed or inherited in different contexts.

```

197     1  class X {
198       2      friend void friend_set (int);
199       3      operator int() {return 0;};
200       4  };
201       5
202       6  void friend_set(int i=X()){} //replaced
203       7
204       8  void f()
205       9  {
206      10      X obj;
207      11      friend_set ();
208      12  }

```

Listing 9. GCC-11XX93, rejects-valid.

```

209
210     1  template <int> struct S{
211       2      friend void foo (int a = []{return 0;}()); //replaced
212       3  };
213       4
214       5  void foo(int a){};
215       6
216       7  int main(){
217       8      S<0> t;
218       9      foo();
219      10  }

```

Listing 10. GCC-11XX19, crash.

A1^X. Super Class Member Importing. Given the classes $cname_1$ and $cname_2$, satisfying $cname_1 \sqsubseteq cname_2 \wedge cname_1 \neq cname_2$, this mutation operator inserts a random importing declaration of $cname_2$ into $cname_1$. This operator introduces a change against the accessibility set, and thus it is not-well-formed.

Listing(11) shows a bug uncovered by this mutator. In this example code, the mutator selects struct D as $cname_1$, A as $cname_2$, and inserts the corresponding importing declaration in line 15, which makes the code invalid with an erroneous overload of struct A's constructor A::A(C). LLVM does not correctly reject that.

A2^X. Global Declaration Importing. Given a class $cname$ and its corresponding program p , this mutation operator inserts a random importing declaration of another global class. This operator may introduce global-level name conflict and thus is not-well-formed.

Listing(12) shows a bug uncovered by this mutator. In this example code, the mutator inserts the friend class declaration in line 4. GCC incorrectly complains about ambiguous name lookup by mixing up the template parameter name `int non_template` and the friend class name.

A5[✓]. Noexcept Function Access. Given a class, this mutation operator inserts noexcept modifier to a member function that has been accessed by the class. This operator is well-formed.

Listing(13) shows a bug uncovered by this mutator. In this example code, the mutator inserts a noexcept modifier to the member function `void S::f()` in line 4, which makes the friend function declaration in line 11 invalid for missing the noexcept specifier. GCC does not realize that and accepts the code.

5 Object Lifetime Mutation Operators

Objects are instances of class types. The instantiation procedure may trigger compiler bugs along the subtyping relationships.

```

1  struct C
2  {
3      C(int = 1){}
4  };
5
6  struct D
7  {
8      D (C);
9  };
10
11 struct A : D
12 {
13     A (const C & n) : D (C{100}) {}
14     using D:: D; //inserted
15 };
16
17 int main(){
18     A a=C{};
19 }

```

Listing 11. LLVM-12XX31, accepts-invalid.

```

1  template <int non_template>
2  struct S {
3      friend class non_template; //inserted
4  };
5
6  class non_template{};
7  S<0> s;

```

Listing 12. GCC-11XX55, rejects-valid.

```

1  struct S
2  {
3      void f()
4      noexcept{} //inserted
5  };
6
7
8  class C
9  {
10     friend void S::f();
11 };
12
13
14 int main()
15 {
16     C ci;
17 }

```

Listing 13. GCC-11XX81, accepts invalid.

O2^X. Ctor Hiding. This mutation operator changes a constructor to private or marks it as deleted equivalent. This operator easily generates invalid programs and is not-well-formed.

Listing(14) shows a bug uncovered by this mutator. In this example code, the mutator modifies the constructor of struct A to private, which disallows struct A to be directly instantiated. But

```

1  struct A{
2  private: //replaced
3      A() {}
4  };
5
6  union C{
7      A a;
8      int b = 0;
9  };
10 int main(){
11     C c;
12 }

```

Listing 14. GCC-11XX28, accepts-invalid.

```

1  struct base {
2      base (int, int) {}
3  };
4
5  template<class>
6  struct derived : base {
7      using base::base;
8      base derived_base_0; //inserted
9      derived() : derived_base_0(42, 42) {} //inserted
10 };
11
12 int main ()
13 {
14     derived<int> (13, 42);
15 }

```

Listing 15. LLVM-13XX13, crash.

when a variable of struct `A` is instantiated inside a union variable, GCC is not able to notice that invalid code.

O6✓. Ctor Interleaving Injection. This mutation operator inserts a correct invocation of another existing class inside the constructor of a class. This operator is well-formed.

Listing 15 shows a bug uncovered by this mutator. In the example code, the mutator inserts the member class of base type `base` in line 9, as well as the corresponding constructor in line 11, which causes LLVM to crash when parsing the inherited constructor.

O7✗. Default Dtor Injection. This mutation operator inserts an explicitly declared destructor. If the destructor is inserted in the private section, the outer invocation of this destructor is disabled, resulting in invalid programs. Thus, this operator is not-well-formed.

Listing(16) shows an accepts-invalid bug uncovered by this mutator. In this example code, a default destructor of class `S` is inserted by the mutator in line 3, making the code invalid, as dynamic initialization of an array object variable needs the destructor to be public. GCC does not correctly realize that.

```
344 1  class S {  
345 2      ~S()=default; //inserted  
346 3  };  
347 4  
348 5  
349 6  int main(){  
350 7      S *ss = new S[100];  
351 8  }
```

Listing 16. GCC-11XX62, accepts-invalid.